

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL - 1 Pseudocode Design

Course Code:	DSA0405	Course Title:	FUNDAMENTALS OF DATA SCIENCE
CO Assessed:	CO5 – Pseudocode Design	Assessment:	ASSESSMENT TOOL - 1 Pseudocode Design
Weightage:	30% of CIA	Total Marks:	100
CO5 Statement:	Design and analyze machine learning workflows using supervised learning, kNN classification, and decision tree techniques for effective data classification and prediction. (BL4)		
Student Name:	MYTHREA SHREE	Reg. No.:	192424089
Section / Batch:	2024	Date:	25.08.2026

Code of Conduct

I LOSHANE S (192424402) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action. Signature of the Student: Loshane S

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly	

Applica on of Concepts	25	Effec vely applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited applica on of concepts	Fails to apply relevant concepts	
Analysis	20	In-depth analysis with strong	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning	

		reasoning and insights				
Solu on Design	20	Innova ve, feasible solu ons with strong jus fica on	Logical solu ons with adequate jus fica on	Basic solu ons with weak jus fica on	Incorrect or no solu on provided	
Clarity	10	Clear, wellstructured, and easy to understand	Organized and understandable presenta on	Limited clarity and structure	Poor clarity; difficult to understand	

Ques on 1 – Supervised Learning

Ques on

Design pseudocode to build a supervised learning workflow that accepts a labelled dataset, separates features and target values, trains a model, and predicts the target for new data.

Problem Statement

A university wants to predict whether a student will pass or fail based on academic performance.

A labelled dataset containing a entrance, internal marks, and assignment scores is available.

The features and target variable are separated from the dataset.

A supervised learning model is trained using the labelled student data. The trained model is then used to predict the result of a new student.

Input

The input dataset is stored in a CSV file named:

student_data.csv

The dataset contains the following columns:

	A	B	C	D	
1	Attendance	Internal_M	Assignmer	Result	
2	90	85	90	Pass	
3	80	75	80	Pass	

The user manually enters the new student's:

- Attendance percentage
- Internal marks
- Assignment score

Procedure / Algorithm

1. Start the program.
2. Read the labelled student dataset from the CSV file.
3. Select Attendance, Internal Marks, and Assignment Score as the input features.
4. Select Result as the target variable.
5. Divide the dataset into training and testing data.
6. Initialize a Decision Tree supervised learning model.
7. Train the model using the training data.
8. Ask the user to manually enter the new student's attendance, internal marks, and assignment score.
9. Store the entered values as a new input record.
10. Use the trained model to predict the student's result.
11. Display the student's entered details and predicted result.
12. Stop the program.

Pseudocode

```
BEGIN

  READ student_data.csv

  X ← Attendance, Internal_Marks, Assignment_Score
  Y ← Result

  SPLIT X and Y into training_data and testing_data

  MODEL ← Initialize Decision Tree Classifier

  TRAIN MODEL using training_data

  INPUT Attendance
  INPUT Internal_Marks
  INPUT Assignment_Score

  new_student ← [Attendance,
                  Internal_Marks,
                  Assignment_Score]

  prediction ← MODEL.PREDICT(new_student)

  DISPLAY Attendance
  DISPLAY Internal_Marks
  DISPLAY Assignment_Score
  DISPLAY prediction

END
```

Practical Implementation

The program reads the CSV file using Pandas and uses a Decision Tree Classifier as the supervised learning model.

```

19]
20]
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

# Read the CSV file
data = pd.read_csv("/content/student_data.csv")

print("Student Dataset:")
print(data)

# Separate features and target
X = data[["Attendance", "Internal_Marks", "Assignment_Score"]]
y = data["Result"]

# Split data into training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create the supervised learning model
model = DecisionTreeClassifier(random_state=42)

# Train the model
model.fit(X_train, y_train)

# New student data
# Get new student details manually
attendance = float(input("Enter Attendance Percentage: "))
internal_marks = float(input("Enter Internal Marks: "))
assignment_score = float(input("Enter Assignment Score: "))

# Create DataFrame for the new student
new_student = pd.DataFrame(
    [[attendance, internal_marks, assignment_score]],
    columns=["Attendance", "Internal_Marks", "Assignment_Score"]
)

# Predict the result
prediction = model.predict(new_student)

print("\nNew Student Details:")
print("Attendance =", attendance, "%")
print("Internal Marks =", internal_marks)
print("Assignment Score =", assignment_score, "%")

print("\nPredicted Result:", prediction[0])

```

Output

```

Student Dataset:
  Attendance  Internal_Marks  Assignment_Score  Result
0         90             85             90      Pass
1         80             75             80      Pass
2         60             45             50      Fail
3         95             90             92      Pass
4         65             50             55      Fail
5         85             80             85      Pass
6         70             55             60      Fail
7         88             82             86      Pass
8         75             65             70      Pass
9         55             40             45      Fail
Enter Attendance Percentage: 90
Enter Internal Marks: 93
Enter Assignment Score: 78

New Student Details:
Attendance = 90.0 %
Internal Marks = 93.0
Assignment Score = 78.0 %

Predicted Result: Pass

```

Result

The supervised learning workflow successfully reads the labelled student dataset from a CSV file, separates the input features and target variable, trains a Decision Tree classification model, accepts new student details manually, and predicts the student's result as Pass.

=====

=====

Ques on 2 – kNN Classifier Ques on

Design pseudocode for a k-Nearest Neighbors (kNN) classifier that calculates the distance between a test sample and all training samples, selects the k nearest samples, and predicts the class using majority voting.

Problem Statement

A retail company wants to classify customers based on their age and annual spending. A labelled customer dataset containing age, annual spending, and customer class is available. The k-Nearest Neighbors algorithm is used to compare a new customer with existing customers.

The distance between the new customer and every training sample is calculated and the nearest customers are selected.

The class occurring most frequently among the selected neighbors is assigned to the new customer.

Input

The input dataset is stored in the CSV file: customer_knn_data.csv

	A	B	C	D
1	Customer	Age	Annual_Sp	Class
2	C1	20	25000	Low
3	C2	22	28000	Low
4	C3	25	32000	Low
5	C4	28	45000	Low
6	C5	30	55000	High
7	C6	32	60000	High
8	C7	35	65000	High
9	C8	38	70000	High
10	C9	40	75000	High
11	C10	42	80000	High

The user manually enters:

- Customer Age
- Annual Spending
- Value of (k) Sample Input

Enter Customer Age: 31

Enter Annual Spending: 58000

Enter value of k: 3

Procedure / Algorithm

1. Start the program.
2. Read the customer dataset from the CSV file.
3. Select Age and Annual Spending as input features.
4. Select Class as the target variable.
5. Get the age and annual spending of the new customer from the user.
6. Get the value of (k).
7. Calculate the distance between the new customer and every customer in the dataset.
8. Store each calculated distance along with its corresponding class.
9. Sort all distances in ascending order.
10. Select the first (k) nearest customers.
11. Count the occurrences of each class among the selected customers.
12. Assign the class with the highest number of votes to the new customer.
13. Display the nearest neighbors and predicted class.
14. Stop.

Distance Formula

For two data points, Euclidean distance is calculated as:

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

where:

- x = Age
- y = Annual Spending

For the new customer:

$$Age = 31$$

$$Annual\ Spending = 58000$$

the distance from each existing customer is calculated.

Pseudocode

```

BEGIN

    READ customer_knn_data.csv

    INPUT Age
    INPUT Annual_Spending
    INPUT k

    distances ← EMPTY LIST

    FOR each customer in dataset DO
        distance ← SQRT(
            (customer.Age - Age)^2 +
            (customer.Annual_Spending - Annual_Spending)^2
        )

        ADD (distance, customer.Class) TO distances
    END FOR

    SORT distances in ascending order

    nearest_neighbors ← FIRST k records from distances

    high_count ← 0
    low_count ← 0

    FOR each neighbor in nearest_neighbors DO
        IF neighbor.Class = "High" THEN
            high_count ← high_count + 1
        ELSE
            low_count ← low_count + 1
        END IF
    END FOR

    IF high_count > low_count THEN
        prediction ← "High"
    ELSE
        prediction ← "Low"
    END IF

    DISPLAY nearest_neighbors
    DISPLAY prediction

END

```

Practical Implementation

```

import pandas as pd
import math

# Read the CSV file
data = pd.read_csv("/content/customer_knn_data.csv")

print("Customer Dataset:")
print(data)

# Manually enter test customer details
age = float(input("\nEnter Customer Age: "))
spending = float(input("Enter Annual Spending: "))

# Select value of k
k = int(input("Enter value of k: "))

# Calculate distance from test customer to every training customer
distances = []

for index, row in data.iterrows():

    distance = math.sqrt(
        (row["Age"] - age) ** 2 +
        (row["Annual_Spending"] - spending) ** 2
    )

    distances.append((distance, row["Class"]))

# Sort distances
distances.sort(key=lambda x: x[0])

# Select k nearest neighbours
nearest = distances[:k]

print("\nNearest Neighbours:")

for distance, class_name in nearest:
    print("Distance =", round(distance, 2), "Class =", class_name)

# Majority voting
classes = [item[1] for item in nearest]

prediction = max(set(classes), key=classes.count)

# Display result
print("\nPredicted Class:", prediction)

```

Sample Execution

```

*** Customer Dataset:
   Customer_ID  Age  Annual_Spending  Class
0           C1   20         25000    Low
1           C2   22         28000    Low
2           C3   25         32000    Low
3           C4   28         45000    Low
4           C5   30         55000   High
5           C6   32         60000   High
6           C7   35         65000   High
7           C8   38         70000   High
8           C9   40         75000   High
9          C10   42         80000   High

Enter Customer Age: 45
Enter Annual Spending: 90000
Enter value of k: 6

Nearest Neighbours:
Distance = 10000.0 Class = High
Distance = 15000.0 Class = High
Distance = 20000.0 Class = High
Distance = 25000.0 Class = High
Distance = 30000.0 Class = High
Distance = 35000.0 Class = High

Predicted Class: High

```

Result

The k-Nearest Neighbors classifier successfully reads the customer data from the CSV file, calculates the distance between the new customer and all existing customers, selects the 6 nearest neighbors, and uses majority voting to classify the new customer.

Thus, the new customer with an age of 45 years and annual spending of ₹90,000 is classified as a High-value customer.

===== Ques on 3 – kNN Classifier Ques on

Design pseudocode to classify a new student as Pass or Fail using kNN based on attendance percentage, internal marks, and assignment scores.

Problem Statement

A university wants to predict whether a new student will pass or fail based on academic performance.

A labelled dataset containing attendance percentage, internal marks, and assignment scores of previous students is available.

The k-Nearest Neighbors algorithm is used to compare the new student with previous students.

The distance between the new student and each training student is calculated, and the nearest students are selected.

The majority class among the nearest students is used to predict whether the new student will Pass or Fail.

Input

The input dataset is stored in: student_knn_data.csv

	A	B	C	D	E
1	Student_ID	Attendance	Internal_M	Assignment	Result
2	S1	90	85	90	Pass
3	S2	85	80	82	Pass
4	S3	88	78	85	Pass
5	S4	70	55	60	Fail
6	S5	65	50	55	Fail
7	S6	72	58	62	Fail

New Student

The program asks the user to manually enter:

Attendance = 87

Internal Marks = 81

Assignment Score = 84

= 3

Procedure / Algorithm

1. Start the program.
2. Read the student dataset from the CSV file.
3. Select Attendance, Internal Marks, and Assignment Score as input features.
4. Select Pass/Fail as the target class.
5. Ask the user to enter the new student's attendance, internal marks, and assignment score.
6. Ask the user to enter the value of (k).
7. Calculate the Euclidean distance between the new student and every student in the dataset.
8. Store each distance with its corresponding Pass/Fail class.
9. Sort the distances in ascending order.
10. Select the first (k) students as the nearest neighbors.
11. Count the number of Pass and Fail classes among the selected neighbors.
12. Assign the class with the majority vote.
13. Display the nearest neighbors and predicted result.
14. Stop.

Distance Formula

For three features, the Euclidean distance is:

$$d = \sqrt{(A_1 - A_2)^2 + (I_1 - I_2)^2 + (S_1 - S_2)^2}$$

where:

- A = Attendance
- I = Internal Marks
- S = Assignment Score

For example, for the new student:

$$A = 87, \quad I = 81, \quad S = 84$$

Pseudocode

```

BEGIN

    READ student_knn_data.csv

    INPUT Attendance
    INPUT Internal_Marks
    INPUT Assignment_Score
    INPUT k

    distances ← EMPTY LIST

    FOR each student in dataset DO

        distance ← SQRT(
            (student.Attendance - Attendance)^2 +
            (student.Internal_Marks - Internal_Marks)^2 +
            (student.Assignment_Score - Assignment_Score)^2
        )

        ADD (distance, student.Result) TO distances

    END FOR

    SORT distances in ascending order

    nearest_students ← FIRST k records

    pass_count ← 0
    fail_count ← 0

    FOR each student in nearest_students DO

        IF student.Result = "Pass" THEN
            pass_count ← pass_count + 1
        ELSE
            fail_count ← fail_count + 1
        END IF

    END FOR

    IF pass_count > fail_count THEN
        prediction ← "PASS"
    ELSE
        prediction ← "FAIL"
    END IF

    DISPLAY nearest_students
    DISPLAY prediction

END

```

Practical Python Implementation

```

import pandas as pd
import math

# Read the CSV file
data = pd.read_csv("../content/student_knn_data.csv")

print("Student Dataset:")
print(data)

# Get new student details manually
attendance = float(input("\nEnter Attendance Percentage: "))
internal_marks = float(input("Enter Internal Marks: "))
assignment_score = float(input("Enter Assignment Score: "))
k = int(input("Enter value of k: "))

# Calculate distances
distances = []

for index, row in data.iterrows():

    distance = math.sqrt(
        (row["Attendance"] - attendance) ** 2 +
        (row["Internal_Marks"] - internal_marks) ** 2 +
        (row["Assignment_Score"] - assignment_score) ** 2
    )

    distances.append(
        (distance, row["Result"])
    )

# Sort distances
distances.sort(key=lambda x: x[0])

# Select k nearest students
nearest = distances[:k]

print("\nNearest Students:")

for distance, result in nearest:
    print("Distance =", round(distance, 2),
          "Result =", result)

# Majority voting
pass_count = 0
fail_count = 0

for distance, result in nearest:

    if result == "Pass":
        pass_count += 1

```

```

        pass_count += 1
    else:
        fail_count += 1

# Predict result
if pass_count > fail_count:
    prediction = "PASS"
else:
    prediction = "FAIL"

print("\nPass Count =", pass_count)
print("Fail Count =", fail_count)

print("\nPredicted Result:", prediction)

```

Sample Execu on

```
*** Student Dataset:
  Student_ID  Attendance  Internal_Marks  Assignment_Score  Result
0          S1         90           85           90      Pass
1          S2         85           80           82      Pass
2          S3         88           78           85      Pass
3          S4         70           55           60      Fail
4          S5         65           50           55      Fail
5          S6         72           58           62      Fail
6          S7         92           88           91      Pass
7          S8         60           45           50      Fail
8          S9         82           72           75      Pass
9         S10         68           52           58      Fail
10        S11         95           92           94      Pass
11        S12         58           42           48      Fail

Enter Attendance Percentage: 87
Enter Internal Marks: 81
Enter Assignment Score: 84
Enter value of k: 3

Nearest Students:
Distance = 3.0 Result = Pass
Distance = 3.32 Result = Pass
Distance = 7.81 Result = Pass

Pass Count = 3
Fail Count = 0

Predicted Result: PASS
```

Result

The kNN classifier successfully reads the labelled student dataset, calculates the distance between the new student and the training students, selects the 3 nearest students, and uses majority voting to predict the student's result.

Predicted Result = PASS

Thus, the new student with 87% attendance, 81 internal marks, and 84 assignment score is classified as Pass.

Ques on 4 – kNN Classifier: Selecting the Best Value of k

Design pseudocode for selecting the appropriate value of k in a kNN classifier by evaluating different k values on validation data.

Problem Statement

A university wants to select the most suitable value of (k) for a kNN student classification model.

Different values of (k) can produce different prediction accuracies.

A labelled student dataset is divided into training and validation data.

The kNN classifier is tested using different values of (k), and its validation accuracy is calculated for each value.

The value of (k) that produces the highest validation accuracy is selected as the best value.

Input

The input dataset is a labelled student dataset containing:

- Attendance
- Internal Marks
- Assignment Score
- Result

Example candidate values of (k):

$$k = 1, 3, 5, 7, 9$$

The dataset is divided into:

- Training Data • Validation Data
-

Procedure / Algorithm

1. Start the program.
2. Read the labelled student dataset from the CSV file.
3. Separate the features and target variable.
4. Split the dataset into training and validation data.
5. Define different candidate values of (k).
6. Select one value of (k).
7. Train the kNN classifier using the training data.
8. Predict the classes of the validation data.
9. Compare the predicted classes with the actual classes.
10. Calculate the validation accuracy.
11. Repeat the process for every candidate value of (k).
12. Compare all validation accuracies.
13. Select the (k) value with the highest accuracy.
14. Display the best (k) and its validation accuracy.

15. Stop.

Accuracy Formula

The validation accuracy is calculated using:

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Predictions}} \times 100$$

For example, if 18 out of 20 validation samples are correctly classified:

$$\text{Accuracy} = \frac{18}{20} \times 100$$

$$\text{Accuracy} = 90\%$$

Pseudocode

```
BEGIN
  READ student_dataset
  X ← Attendance, Internal_Marks, Assignment_Score
  Y ← Result
  SPLIT X and Y into training_data and validation_data
  k_values ← [1, 3, 5, 7, 9]
  best_k ← 0
  best_accuracy ← 0
  FOR each k IN k_values DO
    MODEL ← Initialize kNN with k
    TRAIN MODEL using training_data
    predictions ← MODEL.PREDICT(validation_data)
    accuracy ←
      CORRECT_PREDICTIONS /
      TOTAL_PREDICTIONS × 100
    DISPLAY k and accuracy
    IF accuracy > best_accuracy THEN
      best_accuracy ← accuracy
      best_k ← k
    END IF
  END FOR
  DISPLAY "Best k =", best_k
  DISPLAY "Best Accuracy =", best_accuracy
END
```

Practical Python Implementation

For this question, we can use the same student CSV dataset from Q3.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# Read the CSV file
data = pd.read_csv("/content/student_knn_data.csv")

print("Student Dataset:")
print(data)

# Select features and target
X = data[["Attendance", "Internal_Marks", "Assignment_Score"]]
y = data["Result"]

# Split data into training and validation data
X_train, X_val, y_train, y_val = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=42,
    stratify=y
)

# Different values of k
k_values = [1, 3, 5, 7]

best_k = 0
best_accuracy = 0

print("\nValidation Results:")

for k in k_values:

    # Create KNN model
    model = KNeighborsClassifier(n_neighbors=k)

    # Train model
    model.fit(X_train, y_train)

    # Predict validation data
    predictions = model.predict(X_val)

    # Calculate accuracy
    accuracy = accuracy_score(y_val, predictions) * 100

    print("k =", k, "Accuracy =", round(accuracy, 2), "%")

# Select best k
if accuracy > best_accuracy:
    best_accuracy = accuracy
    best_k = k

print("\nBest k =", best_k)
print("Best Validation Accuracy =",
      round(best_accuracy, 2), "%")
```

Output

Student Dataset:					
	Student_ID	Attendance	Internal_Marks	Assignment_Score	Result
0	S1	90	85	90	Pass
1	S2	85	80	82	Pass
2	S3	88	78	85	Pass
3	S4	70	55	60	Fail
4	S5	65	50	55	Fail
5	S6	72	58	62	Fail
6	S7	92	88	91	Pass
7	S8	60	45	50	Fail
8	S9	82	72	75	Pass
9	S10	68	52	58	Fail
10	S11	95	92	94	Pass
11	S12	58	42	48	Fail

Validation Results:	
k = 1	Accuracy = 100.0 %
k = 3	Accuracy = 100.0 %
k = 5	Accuracy = 100.0 %
k = 7	Accuracy = 100.0 %

Best k = 1
Best Validation Accuracy = 100.0 %

Result

The kNN classifier was evaluated using different values of (k) on the validation dataset. The validation accuracies were compared, and the value of (k) producing the highest validation accuracy was selected.

Thus, (k=1) is selected as the appropriate value for the kNN classifier based on the validation accuracy.

Ques on 5 – Decision Tree Classifier Ques on

Design pseudocode to construct a Decision Tree classifier by selecting the best feature for splitting the training dataset and recursively creating child nodes.

Problem Statement

A university wants to classify students as Pass or Fail based on their academic performance. A labelled dataset containing attendance, internal marks, and assignment scores is available for training.

The Decision Tree algorithm evaluates the available features and selects the best feature for splitting the dataset.

The dataset is recursively divided into smaller child nodes based on the selected features.

The completed Decision Tree is then used to predict the result of a new student.

Input

The input dataset is stored in: student_decision_tree.csv

	A	B	C	D	E
1	Student_ID	Attendance	Internal_M	Assignment	Result
2	S1	90	85	90	Pass
3	S2	85	80	82	Pass
4	S3	88	78	85	Pass
5	S4	70	55	60	Fail
6	S5	65	50	55	Fail
7	S6	72	58	62	Fail

New Student

The program allows the user to manually enter:

Attendance = 87

Internal Marks = 81

Assignment Score = 84

Procedure / Algorithm

1. Start the program.
2. Read the labelled student dataset from the CSV file.
3. Separate the input features and target variable.
4. Select Attendance, Internal Marks, and Assignment Score as features.
5. Select Pass/Fail as the target variable.
6. Initialize a Decision Tree classifier.
7. Calculate the best feature for splitting the training data using a splitting criterion such as Gini Index.
8. Split the dataset based on the selected feature.
9. Repeat the splitting process recursively for each child node.
10. Stop splitting when the stopping condition is reached.
11. Assign the appropriate class to the leaf nodes.
12. Ask the user to enter the details of a new student.
13. Pass the new student data through the Decision Tree.
14. Display the predicted result.
15. Stop.

Gini Index

The Decision Tree can use the Gini Index to select the best split.

Formula:

$$Gini = 1 - \sum p_i^2$$

where (p_i) is the proportion of samples belonging to class (i).

A split with a lower Gini Index provides better class separation.

Pseudocode

```
BEGIN
  READ student_decision_tree.csv
  X ← Attendance, Internal_Marks, Assignment_Score
  Y ← Result
  CREATE Decision_Tree_Model
  FUNCTION BUILD_TREE(data)
    IF all samples belong to the same class THEN
      RETURN Leaf_Node(class)
    END IF
    IF stopping_condition is TRUE THEN
      RETURN Leaf_Node(majority_class)
    END IF
    best_feature ← SELECT_FEATURE_WITH_BEST_SPLIT(data)
    node ← CREATE_NODE(best_feature)
    subsets ← SPLIT data using best_feature
    FOR each subset IN subsets DO
      child_node ← BUILD_TREE(subset)
      ADD child_node TO node
    END FOR
    RETURN node
  END FUNCTION
  tree ← BUILD_TREE(training_data)
  INPUT Attendance
  INPUT Internal_Marks
  INPUT Assignment_Score
  new_student ← [Attendance,
                  Internal_Marks,
                  Assignment_Score]
  prediction ← tree.PREDICT(new_student)
  DISPLAY prediction
END
```

Practical Python Implementation Save the

dataset as: student_decision_tree.csv

Then use the following Python program:

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier

# Read the CSV file
data = pd.read_csv("/content/student_decision_tree.csv")

print("Student Dataset:")
print(data)

# Separate features and target
X = data[
    ["Attendance", "Internal_Marks", "Assignment_Score"]
]

y = data["Result"]

# Create Decision Tree model
model = DecisionTreeClassifier(
    criterion="gini",
    random_state=42
)

# Train the Decision Tree
model.fit(X, y)

# Get new student details manually
attendance = float(
    input("\nEnter Attendance Percentage: ")
)

internal_marks = float(
    input("Enter Internal Marks: ")
)

assignment_score = float(
    input("Enter Assignment Score: ")
)

# Create new student DataFrame
new_student = pd.DataFrame(
    [[attendance, internal_marks, assignment_score]],
    columns=[
        "Attendance",
        "Internal_Marks",
        "Assignment_Score"
    ]
)

# Predict result
prediction = model.predict(new_student)

# Display details
print("\nNew Student Details:")
print("Attendance =", attendance, "%")
print("Internal Marks =", internal_marks)
print("Assignment Score =", assignment_score)

print("\nPredicted Result:", prediction[0])
```

Output

```

Student Dataset:
Student_ID  Attendance  Internal_Marks  Assignment_Score  Result
0           51         90             85              90    Pass
1           52         85             80              82    Pass
2           53         88             78              85    Pass
3           54         70             55              60    Fail
4           55         65             50              55    Fail
5           56         72             58              62    Fail
6           57         92             88              91    Pass
7           58         60             45              50    Fail
8           59         82             72              75    Pass
9           510        68             52              58    Fail
10          511        95             92              94    Pass
11          512        58             42              48    Fail

Enter Attendance Percentage: 87
Enter Internal Marks: 81
Enter Assignment Score: 84

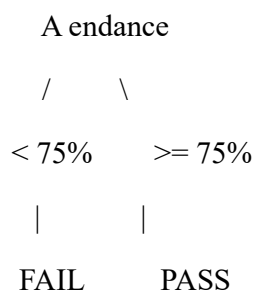
New Student Details:
Attendance = 87.0 %
Internal Marks = 81.0
Assignment Score = 84.0

Predicted Result: Pass

```

Decision Tree Representa on

For this dataset, the trained tree can learn a pa ern similar to:



The exact tree structure is determined automa cally by the Decision Tree algorithm based on the training data and spli ng criterion.

Result

The Decision Tree classifier successfully reads the labelled student dataset, iden fies suitable features for spli ng, recursively constructs the decision tree, accepts new student details, and predicts the student's result.

For the sample student:

- Attendance = 87% • Internal Marks = 81
- Assignment Score = 84 the predicted result is:

PASS